

CSAPP — Cheat Sheet

Computer Systems: A Programmer's Perspective

逐章精炼参考 · 术语/代码保留英文

01 A Tour of Computer Systems

计算机系统漫游 — 全书的路线图：每个主题都会在后续某章展开

• Information = Bits + Context

系统里所有信息——源码、数据、指令——本质都是一串 **bits**。同一段字节序列代表什么，取决于读取它的上下文（当作整数 / 浮点 / 指令 / 字符串）。

`hello.c` 这样的源程序，磁盘上只是一串 ASCII 字节；大多数现代系统用 ASCII / Unicode 表示文本。

• Compilation System 编译系统

`gcc -o hello hello.c` 背后是 4 个阶段，逐步把源码翻译成可执行文件：



预处理展开 `#include / #define`；编译生成汇编；汇编转为二进制目标文件；链接并入 `printf.o` 等库，产生可执行文件。

为何要懂：优化性能、理解链接错误、避开安全漏洞（如缓冲区溢出）。

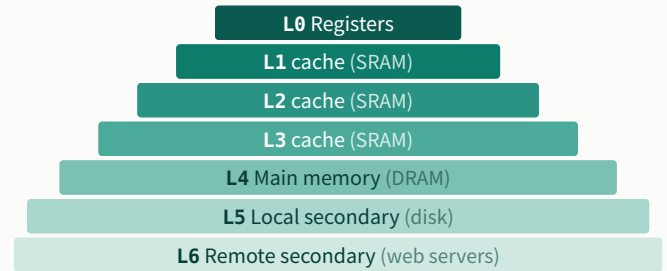
• Hardware Organization 硬件组成

- **CPU**：含 **PC**(程序计数器，指向下条指令)、**register file**(寄存器堆)、**ALU**(算术逻辑单元)。
- **Main memory**：DRAM，临时存放运行中的程序及其数据。
- **Buses**：在部件间传送字节，一次传一个 **word**(64 位机为 8 字节)。
- **I/O devices**：键盘、鼠标、显示器、磁盘，经 controller / adapter 接入。

取指执行循环： CPU 读 PC 所指指令 → 执行 (load / store / operate / jump) → 更新 PC。

• Memory Hierarchy 存储层次

越往上：**更快、更小、每字节更贵**。每一层都是下一层的 **cache**。靠 **locality**(局部性) 起效。



Caches matter: L1–L3 用 SRAM 缓存桥接 CPU 与内存的速度鸿沟。

• The OS & Its Abstractions

OS 介于应用与硬件之间，两大目的：①**保护硬件**不被滥用；②提供**统一**简单的操作接口。它用抽象屏蔽硬件细节：

抽象	抽象了什么
Files	I/O 设备
Virtual memory	主存 + 磁盘 I/O
Processes	处理器 + 主存 + I/O
Virtual machine	整台计算机

• Processes & Threads

- **Process**：OS 制造的“独占机器”假象。多进程靠 **context switch**(上下文切换) 交错执行 → **concurrency**(并发)。
- **Thread**：进程内的多个执行流，**共享代码与全局数据**。比多进程更易共享数据、通常更高效、更利于多核加速。

• Virtual Address Space

每个进程看到一致的地址空间 (x86-64 Linux，高地址在上)：

Kernel virtual memory		用户不可见
User stack	↓ 向下增长	函数调用
Shared libraries		如 <code>libc</code>
Heap	↑ 向上增长	<code>malloc</code>
Read/write data		全局变量
Code (text)		从可执行文件载入

• Concurrency & Parallelism

- **Thread-level**: 多核 / 超线程 (hyperthreading) → 真正同时跑多个线程。
- **Instruction-level (ILP)**: **pipelining**(流水线)、**superscalar**(每周期 >1 条指令)。
- **SIMD**: 单指令处理多数据 (向量运算)。

concurrency(并发) = 逻辑上同时; **parallelism(并行)** = 用并发提升速度。

• Amdahl's Law 阿姆达尔定律

把占比 α 的部分加速 k 倍, 整体加速比:

$$S = \frac{1}{(1 - \alpha) + \alpha / k}$$

极限 $k \rightarrow \infty$ 时 $S = 1 / (1 - \alpha)$ 。

启示: 优化要瞄准**占比大**的部分 (加速常见情形), 否则收益有上限。

• Recurring Themes 贯穿全书的主线

- **Abstractions are key.** ISA 抽象处理器、文件抽象 I/O、虚存抽象内存、进程抽象运行程序。
- **Caches matter.** 理解层次与局部性能带来数量级的性能差异。
- **Networks** 让系统互联——本质上网络也是一种 I/O 设备。